Node.js, Next.js App Router, and TypeScript Strict Mode for Production Web Apps



Rajat Subhra Chakraborty
Professor
Dept. of Computer Science and Engineering
IIT Kharagpur
rschakraborty@cse.iitkgp.ac.in



Agenda & Learning Outcomes

Three connected modules; weights signal teaching emphasis.

M1 · Node.js internals

- Event loop phases and timer semantics
- Streams and backpressure
- Middleware mental model

M2 · TS strict

- Strict mode as a contract
- Generics and discriminated unions

M3 · Next.js App Router

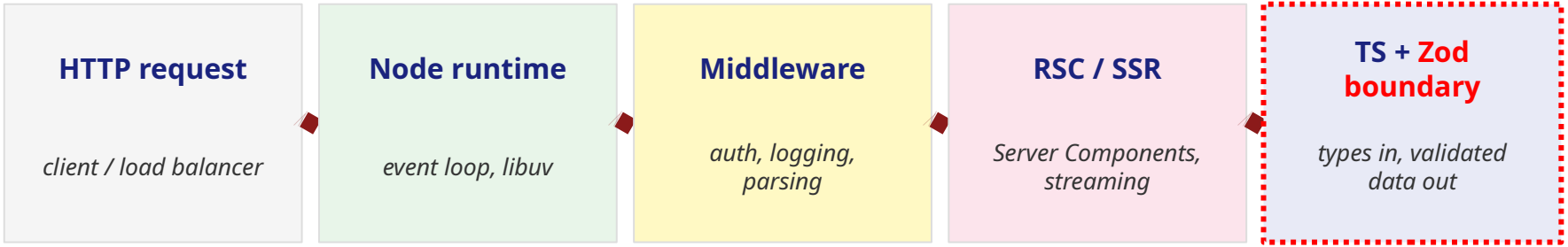
- Server vs Client Components
- Streaming SSR

Learning outcomes

By the end of this session you will be able to: (1) reason about event-loop and stream behavior in production Node services; (2) design APIs and domain models with strict TypeScript (3) build App Router apps that stream, recover from errors, and ship minimal client JavaScript.

A unified mental model

How a request flows through Node, Next.js, and TypeScript.



Where each module fits

- Module 1 — Node internals govern stages 2–3 (event loop, streams, middleware).
- Module 3 — Next.js App Router governs stage 4 (Server vs Client, streaming, errors).
- Module 2 — TypeScript strict + Zod govern stage 5 and the seams between every stage.

Key insight — A request only succeeds if every boundary validates and never blocks the loop.

Event loop phase map

The ***event loop*** is a scheduler at the heart of node.js
It is effectively an **infinite loop** that repeatedly asks, "What callback should I run next?"
Six ordered phases; microtasks run between every phase
The ***event loop*** exists because JavaScript itself is **single-threaded**

1. timers	setTimeout / setInterval callbacks
2. pending callbacks	system callbacks deferred from last iteration (mostly internal)
3. idle, prepare	internal only
4. poll	retrieve new I/O events, may block here
5. check	setImmediate callbacks
6. close callbacks	e.g. socket.on('close')

Microtasks

- process.nextTick queue
- Promise then/catch/finally
- Drained AFTER each phase op
- Can starve I/O if recursive

Timers vs poll vs check

setTimeout, setImmediate, and I/O callbacks live in different phases.

```
// Inside an I/O callback, setImmediate always wins
import fs from 'node:fs';

fs.readFile('a.txt', () => {
  setTimeout(() => console.log('timeout'), 0);
  setImmediate(() => console.log('immediate'));
});
// Output: immediate -> timeout
// Why: we're already past the timers phase
// when the I/O callback fires.
```

setTimeout(fn, 0)

- Runs in timers phase of next iteration
- Threshold ≥ 1 ms internally

setImmediate(fn)

- Runs in check phase, after poll
- Preferred from I/O callbacks

Best practice — Inside any fs/net callback, use setImmediate to defer work to the next loop tick.

process.nextTick and microtask starvation

The nextTick queue runs after the current operation — and before any phase work.

```
// Anti-pattern: recursive nextTick starves I/O
function loop() {
  process.nextTick(loop);   // never yields
}
loop();
// Server stops accepting connections.

// Correct: yield with setImmediate
function tick() {
  doChunk();
  if (more) setImmediate(tick);
}
```

When to use nextTick

- Emit before constructor returns
- Defer error after listener attaches

When NOT to use

- Long-running loops
- Anything that could recurse
- Pacing work — use setImmediate

Key insight — nextTick has higher priority than promises and runs before I/O — easy to starve the loop.

Streams as bounded-memory dataflow

Four stream types, one mental model: chunks flow through a buffer.

Readable	produces data — fs.createReadStream, http.IncomingMessage
Writable	consumes data — fs.createWriteStream, http.ServerResponse
Duplex	both read and write — net.Socket, TLSSocket
Transform	duplex that maps input to output — zlib.createGzip, crypto.Hash

Key insight — Streams turn “memory-shaped” problems (load it all) into “time-shaped” problems (process a chunk at a time).

Streams: pipeline vs pipe and error propagation

A **stream** typically operates on chunks of data (e.g. a large file)

pipe is **convenient**; pipeline is **correct**.

```
import { pipeline } from 'node:stream/promises';
import { createReadStream, createWriteStream } from 'node:fs';
import { createGzip } from 'node:zlib';

await pipeline(
  createReadStream('in.log'),
  createGzip(),
  createWriteStream('in.log.gz'),
);
// Errors propagate. Streams are destroyed on failure.
```

Examples: **“streamDemo.js”** vs. **“streamDemo.ts”**

pipe() pitfalls

- Errors do not propagate
- Source not auto-destroyed
- Manual cleanup required

JS streaming and pipe example (“streamDemo.js”)

pipeline() wins

- Forwards errors
- Destroys all streams on error
- Promise variant integrates with async/await

TS streaming and pipeline example (“streamDemo.ts”)

Middleware chain mental model

A **middleware()** is a function with signature: **(req, res, next) => { ... }**
Typically controls **http requests/response flow**



Four things a middleware can do:

1) Execute code 2) Mutate req/res 3) End the response **4) Call next() to continue**

```
app.use((req, res, next) => {
  req.startedAt = Date.now();
  res.on('finish', () => {
    console.log(req.method, req.url, Date.now() - req.startedAt, 'ms');
  });
  next();   // forget this and the request hangs
});
```

Example: **middlewareExpressServer.ts**

Note: middleware may use streams for data storage!

Strict mode as a production contract

"strict": true enables a family of stronger checks — opt out by name, not by silence.

```jsonc
// tsconfig.json
{
  "compilerOptions": {
    "strict": true,                        // family flag
    "noUncheckedIndexedAccess": true,      // extra, recommended
    "exactOptionalPropertyTypes": true,
    "noImplicitOverride": true
  }
}
```

Strict family flags

- noImplicitAny
- strictNullChecks
- strictFunctionTypes
- strictBindCallApply
- strictPropertyInitialization
- noImplicitThis
- useUnknownInCatchVariables
- alwaysStrict

See Example: **strictDemo.ts**

Generics: preserving relationships without ‘any’

Type variables tie an input type to an output type.

```
// Without generics — relationship lost
function first(arr: any[]): any { return arr[0]; }
const x = first([1, 2, 3]);  // x is any

// With generics — relationship preserved
function first<T>(arr: readonly T[]): T | undefined {
  return arr[0];
}
const y = first([1, 2, 3]);   // y: number | undefined
const z = first(['a', 'b']);  // z: string | undefined
```

Why it matters

- No ‘any’ leaking through
- Inference at call sites
- Refactor safety in shared libs
- Self-documenting APIs

See Example: **genericStack.ts**

Generic constraints with ‘extends’ and ‘keyof’

A “constraint” on a generic type “T”: “**This generic type can be anything, as long as it satisfies certain requirements**."

```
// Constrain to objects with a length
function longest<T extends { length: number }>(a: T, b: T) {
  return a.length >= b.length ? a : b;
}
longest([1, 2], [1, 2, 3]);  // ok
longest('hi', 'world');      // ok
longest(1, 2);               // ✗ no length

// keyof for property access
function pick<T, K extends keyof T>(obj: T, k: K): T[K] {
  return obj[k];
}
```

Constraint vocabulary

- T extends Shape — must include
- keyof T: union of all properties of T
- K extends keyof T — valid key

See Example: **genericConstraintsKeyof.ts**

Discriminated unions

A shared literal property makes the variants distinguishable.

```ts
type Fetch<T> =
  | { status: 'idle' }
  | { status: 'loading' }
  | { status: 'success'; data: T }
  | { status: 'error';   error: Error };

function render(s: Fetch<User>) {
  switch (s.status) {
    case 'idle':    return <Empty />;
    case 'loading': return <Spinner />;
    case 'success': return <Profile user={s.data} />;
    case 'error':   return <Err msg={s.error.message} />;
  }
}
```

Why discriminated unions

- Impossible states become impossible
- switch narrows each case
- Pairs naturally with state machines
- Serializes cleanly over the wire

Exhaustiveness checking with ‘never’

Force the compiler to catch the case you forgot.

```
type Shape =
  | { kind: 'circle'; r: number }
  | { kind: 'square'; side: number }
  | { kind: 'triangle'; base: number; height: number };

function area(s: Shape): number {
  switch (s.kind) {
    case 'circle':   return Math.PI * s.r ** 2;
    case 'square':   return s.side ** 2;
    case 'triangle': return 0.5 * s.base * s.height;
    default: {
      const _exhaustive: never = s;  // errors if a case is missing
      return _exhaustive;
    }
  }
}
```

Key insight — Adding a new variant becomes a compile error everywhere it must be handled — that is the goal.

Why the App Router changes architecture

Server-first rendering with a deliberate client boundary.

Pages Router (legacy)

- One render mode per route
- getServerSideProps / getStaticProps
- All component code shipped to the client
- _app.js wraps everything

App Router

- Server Components by default
- Streaming SSR + Suspense
- Nested layouts that preserve state
- Co-located error and loading UI

What this enables

• Less JavaScript shipped to the browser — Server Components stay on the server.
• Faster first byte — the shell can stream while data still loads.
• Resilience — uncaught errors are contained to a route segment, not the whole app.

File-system routing: folders, page, layout

A folder is a route segment; conventional files define UI.

```
app/
├── layout.tsx          // root layout (REQUIRED, html/body)
├── page.tsx            // /
├── dashboard/
│   ├── layout.tsx      // /dashboard shell
│   ├── page.tsx        // /dashboard
│   ├── loading.tsx     // streaming fallback
│   ├── error.tsx       // route-segment error boundary
│   └── reports/
│       └── page.tsx    // /dashboard/reports
└── (marketing)/        // route group, no URL segment
    └── page.tsx
```

Conventional files

- page — route UI
- layout — wraps children
- loading — Suspense fallback
- error — error boundary
- not-found — 404
- route — REST handler

Layout hierarchy and state preservation

Layouts persist across navigation; pages re-render.

```tsx
// app/dashboard/layout.tsx
export default function DashboardLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <div className="grid grid-cols-[240px_1fr]">
      <Sidebar />          {/* keeps scroll + open state */}
      <main>{children}</main>
    </div>
  );
}
```

Layout rules

- Root layout REQUIRED (html, body)
- Receives children prop only
- No searchParams (use page)
- Stays interactive between routes
- Nest freely along the route tree

Key insight — If state must survive navigation between sibling pages, it belongs in the shared layout.

Pages vs layouts: what belongs where

Use the right convention for what changes per route.

	layout.tsx	page.tsx
URL-bound props	no searchParams	params + searchParams
Re-renders on nav	no — persists	yes — on URL change
Owns route UI	shell / chrome	route content
Children prop	yes	no
Good for	sidebars, tabs, providers	lists, detail views, forms

Best practice — Keep data that changes per URL in page.tsx; keep shell, auth context, and providers in layout.tsx.

Client Components and the 'use client' boundary

The directive marks where the client bundle begins.

```
'use client';
import { useState } from 'react';

export function Counter() {
  const [n, setN] = useState(0);
  return (
    <button onClick={() => setN(n + 1)}>
      Clicked {n} times
    </button>
  );
}
```

When you need it

- useState / useReducer / useEffect
- Event handlers (onClick, onChange)
- Browser APIs (window, localStorage)
- Class components or refs
- Context providers used by client code

Best practice — Push 'use client' as deep into the tree as possible — every import below it enters the client bundle.

Server Components by default

Render on the server, send HTML, ship zero component JavaScript.

```tsx
// app/users/page.tsx — Server Component (no 'use client')
import { db } from '@/lib/db';

export default async function UsersPage() {
  const users = await db.user.findMany();   // direct DB access
  return (
    <ul>
      {users.map(u => <li key={u.id}>{u.name}</li>)}
    </ul>
  );
}
```

Wins

- Smaller client bundle
- Secrets stay server-side
- Async/await in components
- Fetch close to the data

Key insight — Default to Server; opt into Client only when you need browser APIs, state, or event handlers.

Data fetching close to the source

Fetch in the Server Component that uses the data — no prop drilling.

```tsx
// app/orders/[id]/page.tsx
import { getOrder } from '@/lib/orders';
import { Items } from './items';

export default async function OrderPage({
  params: { id },
}: { params: { id: string } }) {
  const order = await getOrder(id);
  return (
    <>
      <header>{order.customer.name}</header>
      <Items orderId={id} />     {/* fetches its own data */}
    </>
  );
}
```

Why this works

- fetch is automatically de-duped per request
- Parallel awaits stream independently
- No need to hoist props through layouts
- Caching is per-fetch, not per-page

Static, dynamic, cached, request-time — a mental ~~model~~

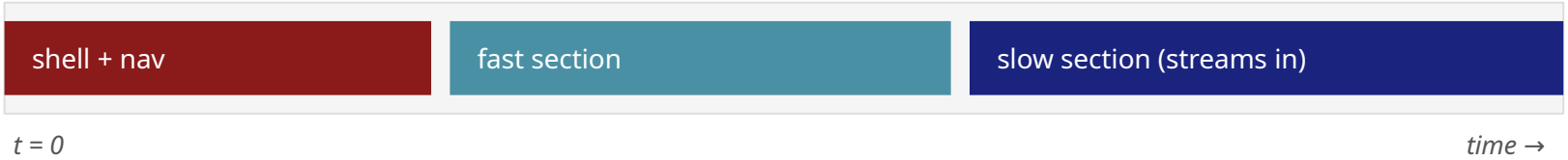
Each route resolves at a specific moment with a specific cache.

Mode	When it runs	Typical trigger
Static	build time	`no dynamic APIs used`
Revalidated	periodically	`fetch with next.revalidate`
On-demand	after revalidate call	`revalidateTag / revalidatePath`
Dynamic	every request	`cookies(), headers(), no-store`

Key insight — Render mode is a function of the APIs you call — not a config flag.

Streaming SSR: why it improves perceived performance

Send the shell early; stream slow segments as they resolve.



What the user sees

- First paint of shell almost immediately
- Spinners only where data is still loading
- No big-bang white screen

What you do

- Wrap slow trees in <Suspense>
- Or add a loading.tsx
- Avoid blocking the shell with slow awaits

Example: **“streaming-demo”**

Key takeaways

1. Respect the event loop — measure delay, never block, use streams for big payloads.
2. Make invariants compile — strict mode + discriminated unions + Result types

Q & A

rschakraborty@cse.iitkgp.ac.in · IIT Kharagpur × upGrad